Shrimand Rajchandra Institute of
Management and Computer
Application, UTU, FYMCA

Cloud Computing with DevOps

Assignment

MCA 2th Semester

2025-26

Gandevia Aqsa
202504104610010
SRIMCA, UTU

INDEX

1. Introduction to cloud computing 1
1.1What is cloud computing ? 1
1.2 The History of cloud computing 1
1.3 Cloud Service model 2
1.3.1 IaaS - Infrastructure as a Service 2
1.3.2 PaaS – Platform as a Service 2
1.3.3 SaaS – Software as a Service 3
1.4 Google Cloud computing services 3
2. Introduction DevOps 5
2.1 What is DevOps ? 5
2.2 How DevOps Works ? 5
2.3 Introduction To Git : 7
2.3.1 What is Git ? 7
2.3.2 Common Git Commands : 7
2.4 All about Jenkins 9
2.4.1. What is Jenkins? 9
2.4.2. Install Jenkins 9
2.4.3.CI/CD Pipeline Flow 10
3.4 What is Docker ? 10
3.4.1 Docker commands : 10
4.4. What is Kubernetes ? 11
4.4.1.Features of Kubernetes 11
4.4.2. Advantages of Kubernetes 11
4.4.3.Important Kubernetes Objects 11

1. Introduction to cloud computing

1.1What is cloud computing ?

- The US National Institute of Standards and Technology created the term cloud computing, although there's nothing Us-specific about it.
- Cloud computing is a way of using that has these five equally important traits.

  1) Customers get computing resources that are on-demand and self-service.

  2) Customers get access to those resources over the internet, from anywhere.

  3) The provider off these resources allocates them to users out of that pool

  4) The resources are elastic-which means they can increase or decrease as needed

  5) customers pay only for what they use, or reserve as they go

1.2 The History of cloud computing

- The trend toward cloud computing started with a first wave known as colocation.
- Second wave known as a Virtualized data centers of today.
- Third wave known as Container-based architecture.

1.3 Cloud Service model

1.3.1 IaaS - Infrastructure as a Service

- IaaS offerings provide raw compute, storage and network capabilities, organized virtually into resources that are similar to physical data centers.

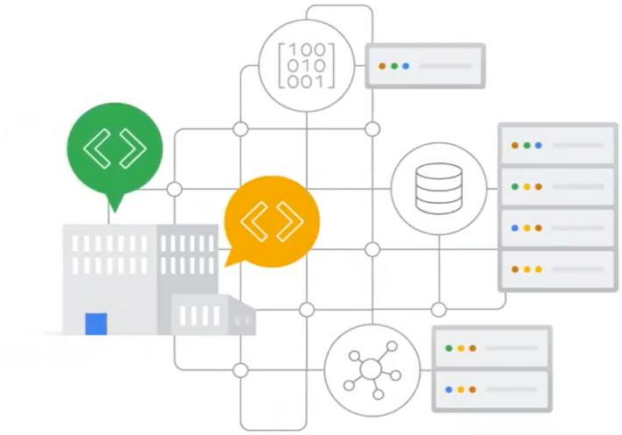
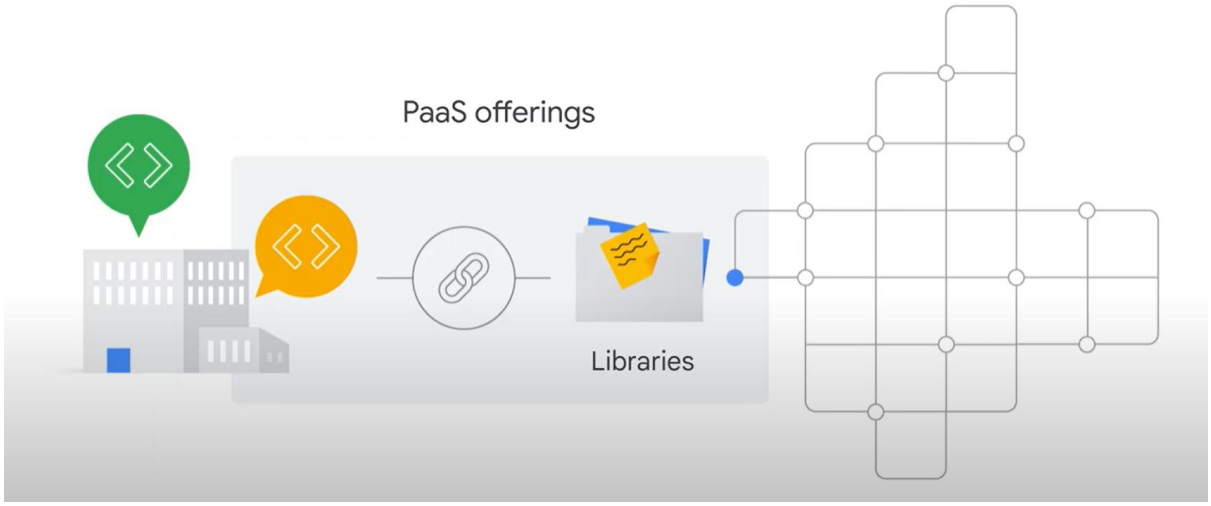


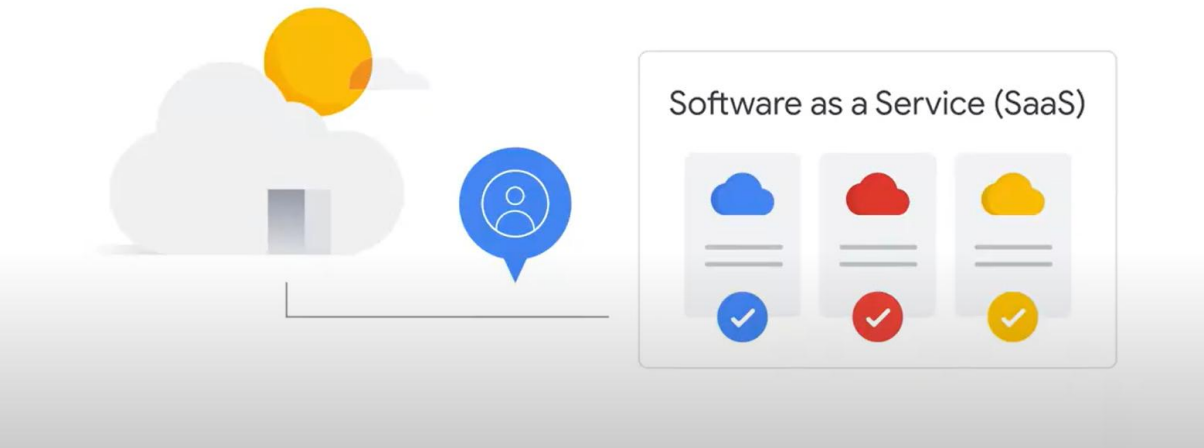
Figure : Infrastructure as Service

1.3.2 PaaS – Platform as a Service



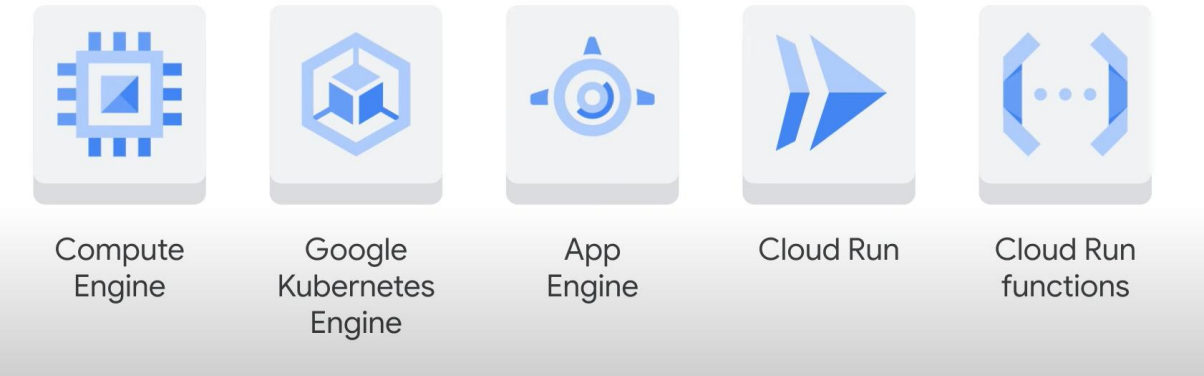
- PaaS offerings bind code to libraries that provide access to the infrastructure applications need.
- This allows more resources to be focused on application logic.

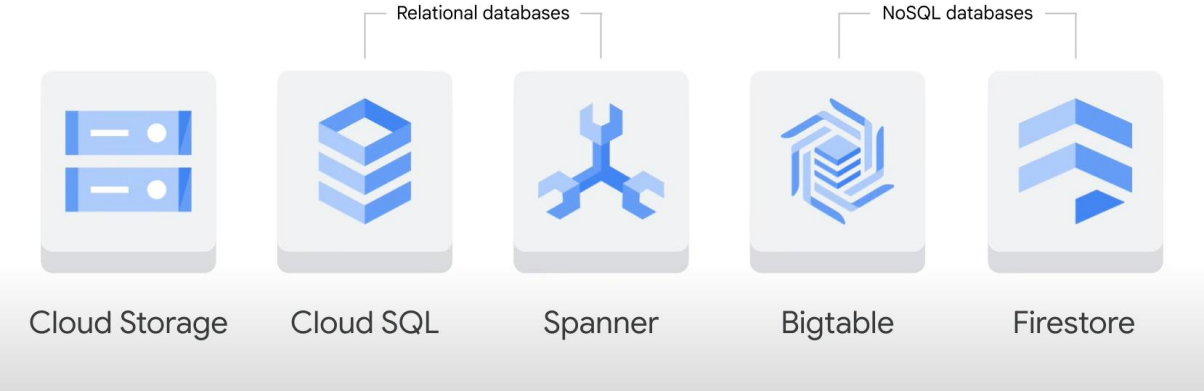
1.3.3 SaaS – Software as a Service



- SaaS applications are not installed on your local computer.
- They run in the cloud as a service and are consumed directly over the internet by end users.
- Google's popular applications like Gmail, Docs, and Drive, collectively known as Google Workspace, are all classified as SaaS.

1.4 Google Cloud computing services




Relational databases
NoSQL databases
Cloud Storage
Cloud SQL
Spanner
Bigtable
Firestore

2. Introduction DevOps

2.1 What is DevOps ?

- DevOps is a software development methodology which improves the collaboration between developers and operations team using various automation tools.
- These automation tools are implemented using various stages which are a part of the DevOps Lifecycle.

Figure : DevOps Lifecycle

2.2 How DevOps Works ?

- The DevOps Lifecycle divides the SDLC lifecycle into the following stages:

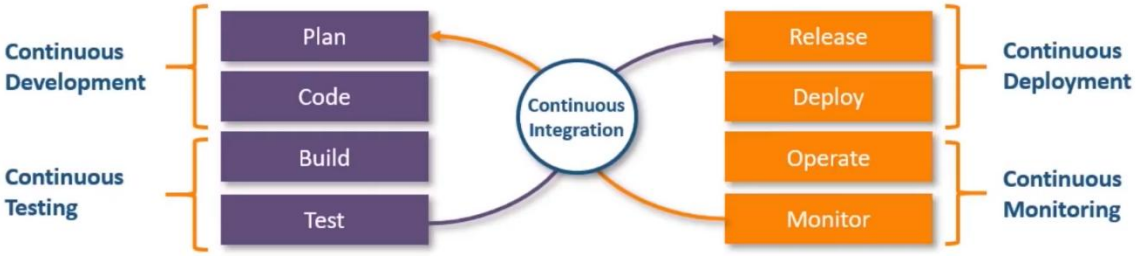


Figure : Stages for DevOps Lifecycle

Continuous development :

- This stage involves committing code to version control tools such as Git or SVN for maintaining the different versions of the code, and tools like Ant, Maven, Gradle for building/packaging the code into an executable file that can be forwarded to the QAs for testing.

Continuous Integration :

- The stage is critical point in the whole DevOps Lifecycle. It deals with integrating the different stages of the devops lifecycle, and is therefore the key in automating the whole DevOps Process.

Continuous Deployment :

- In this stage the code is built, the environment or the application is containerized and is pushed on the desired server. The key processes in this stage are configuration Management, Virtualization and Containerization.

Continuous Testing :

- The stage deals with automated testing of the application pushed by the developer. If there is an error, the message is sent back to the integration tool,this tool in turn notifies the developer of the error. If the test was a success, the message is sent to integration tool which pushes the build on the production server.

Continuous Monitoring :

- The stage continuously monitors the deployed application for bugs or crashes. It can also be setup to collect user feedback. The collected data is then sent to the developers to improve the application.

2.3 Introduction To Git :

- Git is the most popular tool among all the DVCS tools.

2.3.1 What is Git ?

- Git is a version-control system for tracking changes in computer files and coordinating work on those files among multiple people.
- It is primarily used for source-code management in software development, but it can be used to keep track of changes in any set of files.

2.3.2 Common Git Commands :

- Create Repository

- Making Changes

- Parallel Development

- Syncing Repositories

1) git init

- You can create a repository using the command git init. Navigate to your project folder and enter the command **git init** to initialize a git repository for your project on the local system.

2) git status

- Once the directory has been initialized you can check the status of the files, whether they are being tracked by git or not, using the command **git status**

3) git add

- Since no files are being tracked right now, let us now stage these files. For that, enter the command git add. If we want to track all the files in the project folder, we can type the command, **git add .**

4) git commit

- Once the files or changes have been staged, we are ready to commit them in our repository. We can commit the files using the command **git commit -m "Custom message"**

5) git remote

- Once everything is ready on our local, we can start pushing our changes to the remote repository. Copy your repository link and paste it in the command **git remote add origin "URL to repository>"**

6) git push

- To push the changes to your repository, enter the command **git push origin <branch-name>** and hit enter. In our case the branch I master, hence git push origin master
- This command will then prompt for username and password, enter the values and hit enter.

7) git clone

- Similarly, if we want to download the remote repository to our local system, we can use the command **git clone<URL>**. This command will create a folder with the repository name, and download all the contents of the repository inside this folder.

8) git pull

- The git pull command is also used for pulling the latest changes from the repository, unlike git clone, this command can only work inside an initialized git repository.
- This command is used when you are already working in the cloned repository, and want to pull the latest changes, that others might have pushed to the remote repository **git pull <URL of link>**

2.4 All about Jenkins

2.4.1. What is Jenkins?

- **Jenkins** is an open-source automation server used to implement **Continuous Integration (CI)** and **Continuous Deployment (CD)** in DevOps.
- Code building
- Testing
- Deployment
- Monitoring
- Jenkins is written in Java and runs as a web application.

2.4.2. Install Jenkins

Step 1: Install Java

Jenkins requires Java (JDK 8 or higher).

Step 2: Download Jenkins

From official website.

Step 3: Start Jenkins

Runs on:

http://localhost:8080

Step 4: Unlock Jenkins

- Copy initial admin password
- Install suggested plugins
- Create admin user

2.4.3.CI/CD Pipeline Flow

1. Developer writes code
2. Push to GitHub
3. Jenkins triggers build
4. Run unit tests
5. Create build artifact
6. Build Docker image
7. Deploy to server

Tools commonly integrated:

- Docker
- Kubernetes
- Maven

3.4 What is Docker ?

- Application containerization is an OS-level virtualization method used to deploy and run distributed applications without launching an entire virtal machine foe each app

3.4.1 Docker commands :

- **docker pull <image-name>**

  This command helps you pull images from the central docker repository.

- **docker ps**

  This command helps in listing all the containers which are running in the system

- **docker ps -a**

  If there are any stopped containers , they can be seen by adding the "-a" flag in this command.

- **docker exec <container-id>**

  For logging into /accessing the container, one can use the exec command

4.4. What is Kubernetes ?

Kubernetes (often called **K8s**) is an open-source container orchestration platform used to deploy, manage, and scale containerized applications automatically. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation. Kubernetes helps manage containers created using tools like Docker and ensures applications run reliably across clusters of machines.

4.4.1.Features of Kubernetes

- Automatic scaling (Horizontal Pod Autoscaler)
- Self-healing (Restart failed containers)
- Load balancing
- Rolling updates & rollbacks
- Secret & configuration management

4.4.2. Advantages of Kubernetes

- Highly scalable
- Portable across cloud providers
- High availability
- Efficient resource utilization
- Large community support

4.4.3.Important Kubernetes Objects

➢ **Pod**

Smallest deployable unit.

- Contains one or more containers.
- Shares network and storage.

➢ **Deployment**

- Manages pods.

- Ensures desired number of replicas.
- Supports rolling updates.

➢ **Service**

- Exposes pods to internal or external users.
- Provides stable IP address.

➢ **ReplicaSet**

- Maintains specified number of pod replicas.

➢ **Namespace**

- Divides cluster into logical groups.